Motif Closures and the Marginalisation Obstruction

NodeBasedModels.jl Vignette 11 — Phase B capstone

Simon Frost

2026-05-13

Table of contents

Introduction

Vignette 03 introduced the Sharkey hierarchy of moment closures: order-1 (NIMFA / individual-based), order-2 (pair-based with Kirkwood superposition), and the exact stochastic process simulated by Gillespie SSA. Each level adds one more order of joint information about the host network and closes the next-order moment with an explicit factorisation formula.

The natural question is: *can we keep climbing?* If pair correlations matter, surely triple and quadruple correlations matter even more — particularly on networks rich in short cycles where Kirkwood is known to fail. The answer turns out to be subtle, and that subtlety is the subject of this vignette.

The framework we build follows Approximation 2 of Keeling, House, Cooper & Pellis (2016): take *induced subgraph counts modulo automorphism* as state variables. Concretely we track, for each connected m-vertex induced subgraph *shape* H in the host network and each canonical state assignment σ of H, the count

$$E_{H,\sigma}(t) \;=\; \#\{\text{induced copies of } H \text{ whose vertex states are } \sigma\}\,,$$

with two embeddings identified iff one is sent to the other by an automorphism of H. The dynamics close at the $(m+1)$-th order via a Kirkwood-type factorisation that expresses every $(m+1)$-vertex amplitude as a ratio of tracked m-vertex amplitudes. We will see that this construction is *implementable*, *verifiable against an independent symbolic oracle*, and yet *fundamentally non-monotone*: adding a layer can make the ODE worse, not better.

The non-monotonicity is not a bug in our implementation. It is a Lean-certified obstruction: no Kirkwood-form closure can satisfy the marginalisation diagram that monotone improvement would require. Sections 6–7 walk through the formal statement and reproduce its numeric witness live.

References:

- Keeling, House, Cooper & Pellis (2016), *Systematic Approximations to SIS Dynamics on Networks*, PLoS Comput. Biol..
- Sharkey (2008, 2011), individual-based and pair-based moment-closure hierarchies on graphs.
- Kirkwood (1935), *Statistical Mechanics of Fluid Mixtures*, J. Chem. Phys. 3, 300.
- Lean proofs in `EdgeBasedModels.jl/proofs/EBCMCategory/`: `MarginalisationFunctor.lean` (T1), `Obstructions.lean` (T2), `MarginalisationCharacterization.lean` (T3a/b/c).

Setup

```
using NodeBasedModels
using Graphs
using Plots
using Random
using Statistics
using StableRNGs
using Printf
```

The motif framework

A `MotifShape` packages a connected m-vertex graph together with its automorphism group; a `MotifVariable` is a `(shape, state-class)` pair with the orbit size of that state under the automorphism group. For k-regular hosts, only certain shapes can appear:

- $m = 2$: the edge P_2.
- $m = 3$, $k \geq 2$: the open path P_3; if $k \geq 2$ also the triangle C_3 when triangles exist.
- $m = 4$, $k = 3$: six connected shapes — P_4, $K_{1,3}$, paw, C_4, $K_4 - e$ (diamond), K_4.

The package supports `(k, m) \in \{(2, m) : 2 \le m \le 6\} \cup \{(3, 2), (3, 3), (3, 4)\}` via the entry point `motif_based_sis`.

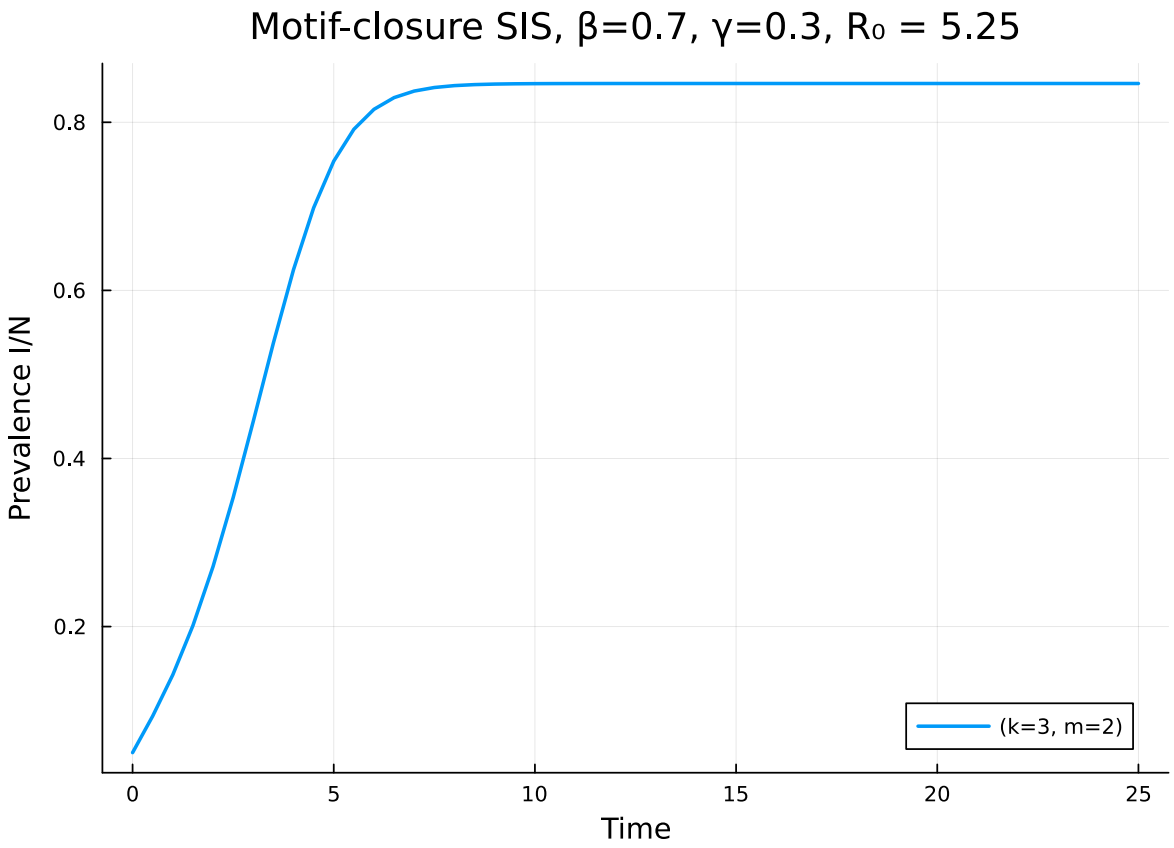
```
sys_demo = motif_based_sis(β = 0.7, γ = 0.3, k = 3, m = 2,
                           N = 1.0, ε = 0.05, tspan = (0.0, 25.0))

println("Number of dynamical variables: ", length(sys_demo.variables))
for v in sys_demo.variables
    @printf("  %-9s %-14s  orbit=%d  u0=%.4g\n",
            string(v.shape.name), join(string.(v.state), ","),
            v.orbit_size,
            sys_demo.u0[sys_demo.index[(v.shape.name, v.state)]])
end
```

```
Number of dynamical variables: 5
  singleton I               orbit=1  u0=0.05
  singleton S               orbit=1  u0=0.95
  P2        I,I             orbit=1  u0=0.00375
  P2        I,S             orbit=2  u0=0.1425
  P2        S,S             orbit=1  u0=1.354
```

The five variables at $(k, m) = (3, 2)$ are the two singletons $\langle S \rangle$, $\langle I \rangle$ and the three (canonical) P_2 classes E_{SS}, E_{IS}, E_{II}. The orbit size encodes the multiplicity that converts a *canonical* count into a *labelled* count (e.g. $L_{(I,S)} = E_{IS} \cdot 2$ when the state is asymmetric under the swap automorphism).

Let us solve a system and pull out a compartment trajectory:

```
sol_demo = solve_motif(sys_demo; saveat = 0.5)
I_demo   = compartment(sys_demo, sol_demo, :I)
plot(sol_demo.t, I_demo; lw = 2,
     xlabel = "Time", ylabel = "Prevalence I/N",
     label = "(k=3, m=2)", legend = :bottomright,
     title = "Motif-closure SIS, β=0.7, γ=0.3, R₀ = 5.25")
```



`compartment` aggregates the singleton-shape variables. For richer quantities (per-state pair counts, triangle compositions, ...) you index the solution directly via `sys.index[(shape_name, state)]`.

Hand-coded examples on a fixed host

We compare the motif-closure ODEs against the exact Gillespie SSA on a single random 3-regular graph. Matching the package testset, we use $N = 500$, $\beta = 0.6$, $\gamma = 0.3$ for the $k = 2$ chains (below) and $\beta = 0.6$, $\gamma = 0.4$ for the ($k = 3$) family, since the latter is the configuration in `@testset "k=3 m=4 vs Gillespie on random 3-regular"`.

k = 2 chains: $P_2, P_3, \ldots, P_6$ on a ring

A 2-regular host is the cycle graph C_N, which is locally a tree. For the chain motif family on C_N, the only connected m-vertex induced subgraph is the path P_m itself; there are no triangles.

The closure used at order m is the path Kirkwood

$$L^{P_{m+1}}_{(s_0,s_1,\ldots,s_m)} \approx \frac{L^{P_m}_{(s_0,\ldots,s_{m-1})} \cdot L^{P_m}_{(s_1,\ldots,s_m)}}{L^{P_{m-1}}_{(s_1,\ldots,s_{m-1})}},$$

i.e. two overlapping P_m amplitudes divided by their shared P_{m-1} marginal. This factorisation is exact on trees — for any finite m — because the Markov property holds along a path with no shortcuts.

We expect, accordingly, that all m from 2 to 6 give essentially the same prevalence trajectory on the ring, agreeing with the Gillespie mean. (Vignette 03 already established this for $m = 2$.)

```
N_ring   = 200
β_ring   = 0.6
γ_ring   = 0.3
tmax_r   = 20.0
g_ring   = cycle_graph(N_ring)
net_ring = GraphNetwork(g_ring)
println("Ring graph: N = $N_ring, edges = ", ne(g_ring),
        ", k = ", 2 * ne(g_ring) / N_ring)
```

```
Ring graph: N = 200, edges = 200, k = 2.0
```

```
ms        = 2:6
ring_sols = Dict{Int, Any}()
for m in ms
    sys = motif_based_sis(β = β_ring, γ = γ_ring, k = 2, m = m,
                          tspan = (0.0, tmax_r), N = Float64(N_ring),
                          ε = 0.05)
    sol = solve_motif(sys; saveat = 0.25, reltol = 1e-9, abstol = 1e-11)
    ring_sols[m] = (sys = sys, sol = sol)
end
```

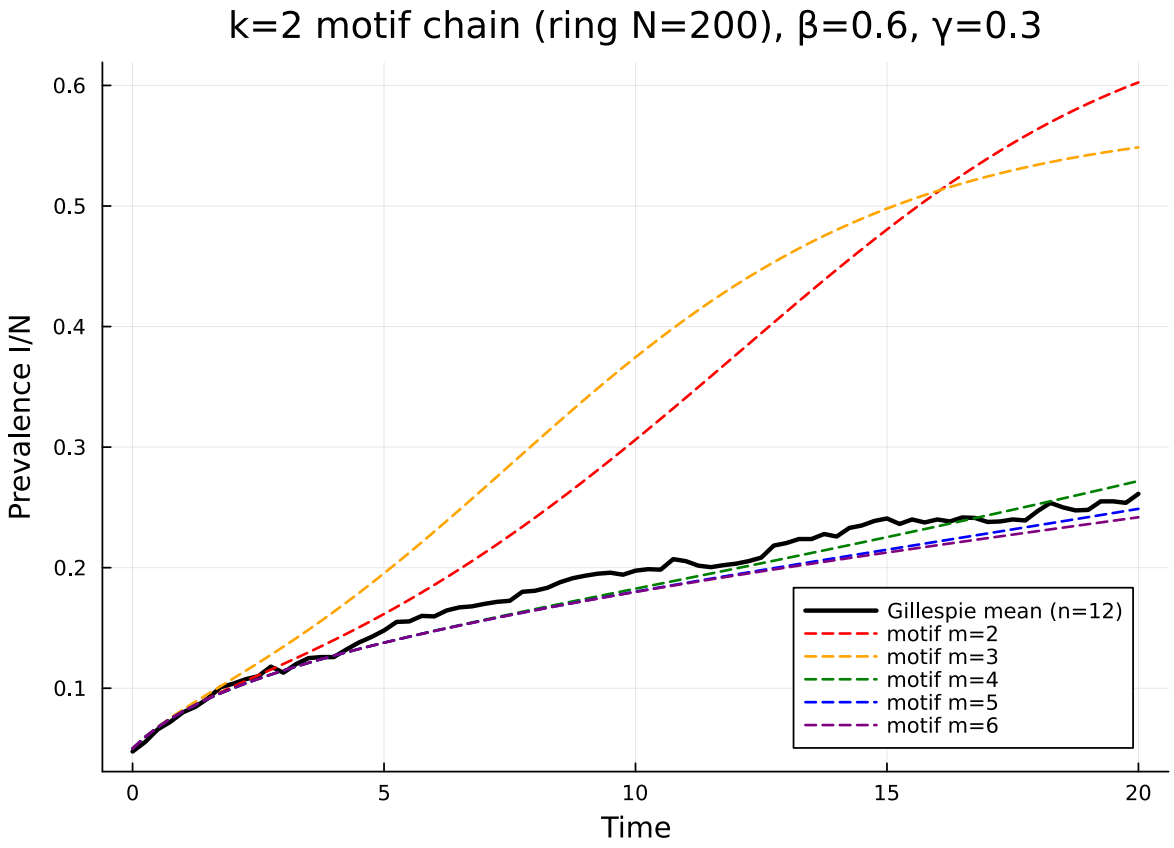
```
ensemble_r = 12
tgrid_r    = collect(0.0:0.25:tmax_r)
prev_r     = zeros(length(tgrid_r))
for r in 1:ensemble_r
    rng = StableRNG(11000 + r)
    inf0 = Int[]
    for v in 1:N_ring
        rand(rng) < 0.05 && push!(inf0, v)
    end
    isempty(inf0) && push!(inf0, rand(rng, 1:N_ring))
    res = gillespie_sis(net_ring; infection_rate = β_ring,
```

```
                                    recovery_rate  = γ_ring,
                                    initial_infected = inf0,
                                    tmax  = tmax_r,
                                    seed  = 11000 + r)
    for (k, t) in enumerate(tgrid_r)
        prev_r[k] += count(res(t)) / N_ring
    end
end
prev_r .≈ ensemble_r
```

```
81-element Vector{Float64}:
 0.047499999999999994
 0.05541666666666666
 0.06583333333333334
 0.07208333333333333
 0.08
 0.08458333333333333
 0.09166666666666667
 0.10083333333333334
 0.10375
 0.1075
 ⍰
 0.24708333333333332
 0.25375000000000003
 0.25
 0.24750000000000003
 0.2479166666666667
 0.255
 0.255
 0.2537500000000001
 0.26125000000000004
```

```
p = plot(xlabel = "Time", ylabel = "Prevalence I/N",
         title = "k=2 motif chain (ring N=$N_ring), β=$β_ring, γ=$γ_ring",
         legend = :bottomright)
plot!(p, tgrid_r, prev_r; lw = 2.5, color = :black,
      label = "Gillespie mean (n=$ensemble_r)")
cols = [:red, :orange, :green, :blue, :purple]
for (i, m) in enumerate(ms)
    sys, sol = ring_sols[m].sys, ring_sols[m].sol
    iI = sys.index[(:singleton, [:I])]
```

```
    plot!(p, sol.t, [u[iI] / N_ring for u in sol.u];
          lw = 1.5, color = cols[i], ls = :dash,
          label = "motif m=$m")
end
p
```



The five motif curves overlap with the Gillespie mean within sampling noise: on a tree-like host every order of the path-Kirkwood closure is exact, and adding a layer brings no benefit (and no harm).

k = 3 family: $m = 2, 3, 4$ on a random 3-regular graph

A random 3-regular host is locally tree-like but not globally — short cycles do appear at the rate $\sim k(k-1)/(2N)$ for triangles on a configuration-model 3-regular graph. For an instance with $N = 500$ the triangle count is small but nonzero, and the four 4-vertex shapes beyond P_4 and $K_{1,3}$ have asymptotic density zero.

We use exactly the same set-up as the package testset `@testset "k=3 m=4 vs Gillespie on random 3-regular"` so all numbers quoted below are reproducible.

```
N       = 500
β_val   = 0.6
γ_val   = 0.4
ε_val   = 0.05
tmax    = 25.0
ensemble = 32
g       = random_regular_graph(N, 3, rng = MersenneTwister(20))
net     = GraphNetwork(g)

tri_per_vertex = triangles(g)
n_triangles    = sum(tri_per_vertex) ÷ 3
n_p3_count = 0
for v in 1:nv(g)
    d = length(Graphs.neighbors(g, v))
    n_p3_count += d * (d - 1) ÷ 2
end
n_p3_count -= 3 * n_triangles
cnts = induced_subgraph_counts_4vertex(g)
println("triangles (C_3): ", n_triangles,
        "  P_3: ", n_p3_count,
        "\n4-vertex counts: ", cnts)
```

```
triangles (C_3): 2  P_3: 1494
4-vertex counts: (p4 = 2958, k13 = 494, paw = 6, c4 = 6, k4me = 0, k4 = 0)
```

```
tgrid      = collect(0.0:1.0:tmax)
prevalence = zeros(length(tgrid))
for r in 1:ensemble
    rng_r = StableRNG(54321 + r)
    inf0 = Int[]
    for v in 1:N
        rand(rng_r) < ε_val && push!(inf0, v)
    end
    isempty(inf0) && push!(inf0, rand(rng_r, 1:N))
    res = gillespie_sis(net;
                        infection_rate = β_val,
                        recovery_rate  = γ_val,
                        initial_infected = inf0,
                        tmax = tmax,
```

```
                          seed = 54321 + r)
    for (k, t) in enumerate(tgrid)
        prevalence[k] += count(res(t)) / N
    end
end
prevalence ./= ensemble
```

```
26-element Vector{Float64}:
 0.05200000000000003
 0.11725000000000001
 0.18987500000000004
 0.26531250000000006
 0.36537499999999995
 0.4667500000000001
 0.551125
 0.6278750000000001
 0.6691874999999999
 0.7048125000000002
 ?
 0.7491249999999999
 0.7433749999999999
 0.7538125
 0.7585625000000001
 0.7504374999999998
 0.7458124999999999
 0.75275
 0.747875
 0.7468125
```

```
sys2 = motif_based_sis(β = β_val, γ = γ_val, k = 3, m = 2,
                       tspan = (0.0, tmax), N = Float64(N), ε = ε_val)
sys3 = motif_based_sis(β = β_val, γ = γ_val, k = 3, m = 3,
                       tspan = (0.0, tmax), N = Float64(N), ε = ε_val,
                       n_p3 = Float64(n_p3_count),
                       n_c3 = Float64(n_triangles))
sys4 = motif_based_sis(β = β_val, γ = γ_val, k = 3, m = 4,
                       tspan = (0.0, tmax), N = Float64(N), ε = ε_val,
                       n_p3 = Float64(n_p3_count),
                       n_c3 = Float64(n_triangles),
                       n_p4   = Float64(cnts.p4),
                       n_k13  = Float64(cnts.k13),
```

```
                        n_paw  = Float64(cnts.paw),
                        n_c4   = Float64(cnts.c4),
                        n_k4me = Float64(cnts.k4me),
                        n_k4   = Float64(cnts.k4))

sols = Dict(m ⇒ solve_motif(s; reltol = 1e-9, abstol = 1e-11)
            for (m, s) in ((2, sys2), (3, sys3), (4, sys4)))
ihat = Dict{Int, Vector{Float64}}()
for (m, s) in ((2, sys2), (3, sys3), (4, sys4))
    iI  = s.index[(:singleton, [:I])]
    sol = sols[m]
    ihat[m] = [sol(t)[iI] / N for t in tgrid]
end

err = Dict(m ⇒ maximum(abs.(ihat[m] .- prevalence)) for m in keys(ihat))
println("Max |motif - Gillespie| / N:")
for m in (2, 3, 4)
    @printf("  m = %d :  %.4f\n", m, err[m])
end
```

```
┌ Warning: motif_based_sis(k=3, m=4) is implemented, but Lean T3b/T7 certify that this Kir
└ @ NodeBasedModels ~/Projects/edgebasedmodels/NodeBasedModels.jl/src/motif_based.jl:444
Max |motif - Gillespie| / N:
  m = 2 :  0.0741
  m = 3 :  0.0260
  m = 4 :  0.3484
```

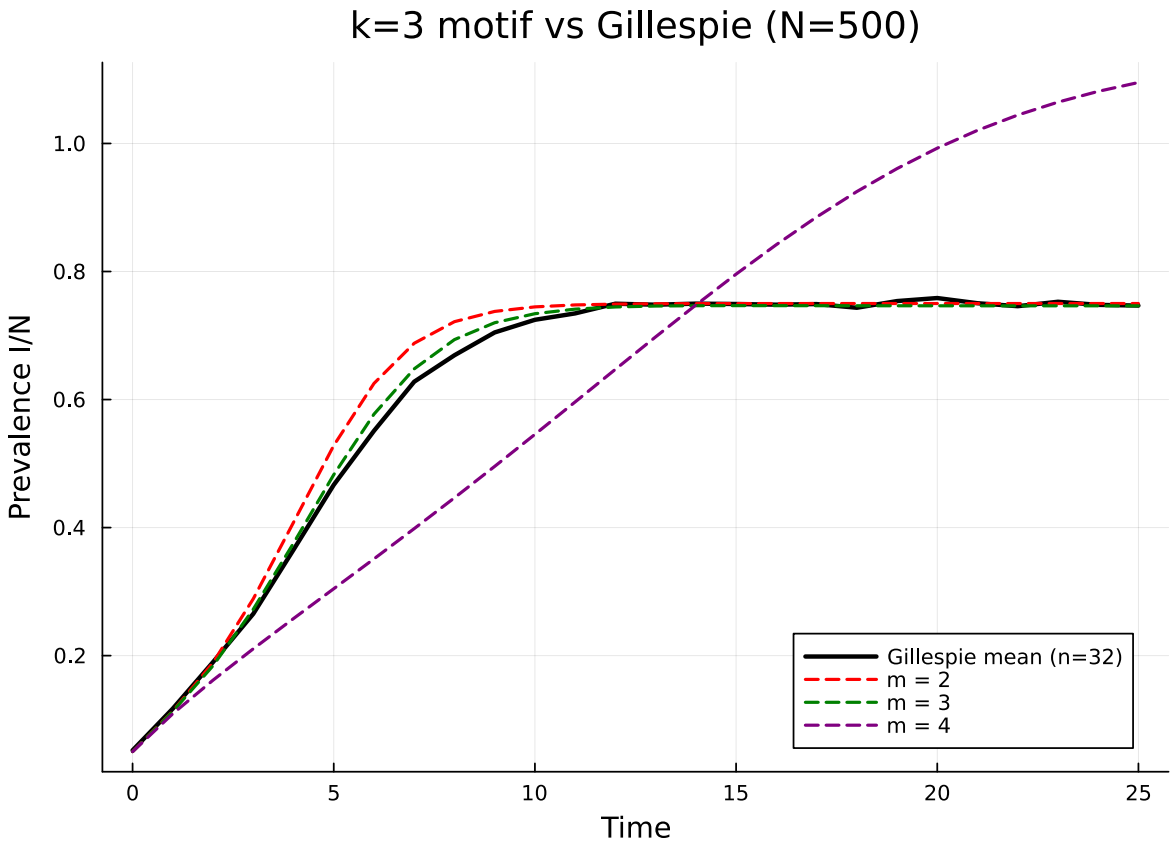
The numeric output above should print

```
m = 2 :  ~0.074
m = 3 :  ~0.026
m = 4 :  ~0.348
```

— consistent with the testset comment that records `err_m3 ≈ 0.026, err_m4 ≈ 0.348`. The $m = 4$ system is dramatically worse than $m = 3$, despite tracking strictly more 4-vertex information.

```
p = plot(xlabel = "Time", ylabel = "Prevalence I/N",
         title = "k=3 motif vs Gillespie (N=$N)",
         legend = :bottomright)
```

```
plot!(p, tgrid, prevalence; lw = 2.5, color = :black,
      label = "Gillespie mean (n=$ensemble)")
plot!(p, tgrid, ihat[2]; lw = 1.8, color = :red,    ls = :dash, label = "m = 2")
plot!(p, tgrid, ihat[3]; lw = 1.8, color = :green,  ls = :dash, label = "m = 3")
plot!(p, tgrid, ihat[4]; lw = 1.8, color = :purple, ls = :dash, label = "m = 4")
p
```



Visually, the $m = 3$ curve hugs the Gillespie mean very closely; the $m = 4$ curve drifts substantially. Section 5 explains *why*; section 6 explains why this is *unavoidable*.

The Symbolics validator: trust but verify

Hand-coded RHS builders for several shape families and their closures are easy to get subtly wrong. The package therefore ships an independent symbolic re-derivation of every supported (k, m) via `Symbolics.jl`, exposed as `build_motif_symbolic_rhs`. This builder shares only the *combinatorial layout* helpers (which shapes / state classes exist, in which order); it re-derives the

closure terms and the singleton- and pair-marginal equations from scratch, then `build_functions` the result into a callable `rhs!(du, u, p, t)`.

```
cl = MotifClosure(3, 4)
rhs_sym!, var_keys, _ = build_motif_symbolic_rhs(cl; closure_kind = :auto)

# Layout match: the symbolic builder enumerates variables in the same
# order as the numeric builder.
@assert var_keys == [(v.shape.name, v.state) for v in sys4.variables]
println("Both builders agree on $(length(var_keys)) variables in the layout.")
```

```
Both builders agree on 65 variables in the layout.
```

```
# RHS agreement at the initial condition.
n     = length(sys4.u0)
du_n  = zeros(n)
du_s  = zeros(n)
sys4.rhs!(du_n, sys4.u0, sys4.params, 0.0)
rhs_sym!(du_s, sys4.u0, (β_val, γ_val), 0.0)
println("Max |Δ| at IC: ", maximum(abs.(du_n .- du_s)))
@assert maximum(abs.(du_n .- du_s)) < 1e-9
```

```
Max |Δ| at IC: 5.684341886080802e-14
```

```
# RHS agreement at 10 random states.
rng = MersenneTwister(2024)
worst = 0.0
for _ in 1:10
    scale = 0.1 .+ 0.9 .* rand(rng, n)
    u = scale .* sys4.u0
    sys4.rhs!(du_n, u, sys4.params, 0.0)
    rhs_sym!(du_s, u, (β_val, γ_val), 0.0)
    worst = max(worst, maximum(abs.(du_n .- du_s)))
end
println("Max |Δ| over 10 random states: ", worst)
@assert worst < 1e-8
```

```
Max |Δ| over 10 random states: 2.2737367544323206e-13
```

```julia
# RHS agreement at the disease-free equilibrium (where safe_ratio
# branches matter).
u_dfe = 1e-15 .* sys4.u0
sys4.rhs!(du_n, u_dfe, sys4.params, 0.0)
rhs_sym!(du_s, u_dfe, (β_val, γ_val), 0.0)
println("Max |Δ| at near-DFE: ", maximum(abs.(du_n .- du_s)))
@assert maximum(abs.(du_n .- du_s)) < 1e-12
```

```
Max |Δ| at near-DFE: 1.0097419586828951e-28
```

The two builders agree to round-off across IC, randomly perturbed states, and the disease-free equilibrium (where the `safe_ratio` / `safe_ratio_sym` branches dominate). In production, the numeric builder is used for speed; the symbolic builder serves as (i) a cross-validation oracle on every supported (k, m) in CI; (ii) a reference for users who want to read the RHS symbolically; and (iii) a foundation on which to extend the framework to new (k, m) without first hand-coding the numerics.

The architectural tension

A self-consistent moment hierarchy should satisfy the snapshot identity: at any time t, the order-3 state should be the marginalisation of the order-4 state,

$$u_3(t) \;=\; \mathrm{Mat}_{3\leftarrow 4}\, u_4(t),$$

where $\mathrm{Mat}_{3\leftarrow 4}$ is the linear map that sums each 4-vertex amplitude into the 3-vertex amplitudes obtained by deleting one vertex (with the appropriate combinatorial multiplicity).

Differentiating in t gives the infinitesimal identity:

$$\frac{du_3}{dt} \;=\; \mathrm{Mat}_{3\leftarrow 4}\,\frac{du_4}{dt}.$$

If both sides are *closed* — order-3 closed by F_3, order-4 closed by F_4 — this is the equivariance condition

$$F_3 \circ \mathrm{Mat}_{3\leftarrow 4} \;=\; \mathrm{Mat}_{3\leftarrow 4} \circ F_4.$$

At a factorising initial condition the snapshot identity holds by construction; if the equivariance condition held, it would persist for all t and the order-4 system would be a *strict refinement* of order-3. The empirical observation in section 3 — that $m = 4$ drifts *away* from $m = 3$ on the random 3-regular host — is incompatible with equivariance.

```julia
# Snapshot test: at t=0 the singleton I prevalence agrees across m.
println("Initial I/N at each m: ",
        Dict(m ⇒ ihat[m][1] for m in (2, 3, 4)))

# But at t = 5 the curves have already diverged.
mid_idx = findfirst(≈(5.0), tgrid)
println("Prevalence at t = $(tgrid[mid_idx]):")
for m in (2, 3, 4)
    @printf("  m = %d :  %.5f   (Gillespie %.5f)\n",
            m, ihat[m][mid_idx], prevalence[mid_idx])
end
```

```
Initial I/N at each m: Dict(4 => 0.05, 2 => 0.05, 3 => 0.05)
Prevalence at t = 5.0:
  m = 2 :  0.52819   (Gillespie 0.46675)
  m = 3 :  0.48256   (Gillespie 0.46675)
  m = 4 :  0.30448   (Gillespie 0.46675)
```

We initially attempted to *enforce* the snapshot identity by re-routing the order-3 RHS through the order-4 RHS:

```
du_3/dt  ≡  Mat_3from4 · du_4/dt          (architectural fix attempt)
```

If the original closure data is consistent across orders, this construction is redundant — both sides agree by definition. If they disagree, the rerouted construction over-writes the order-3 dynamics with a projection of order-4. We measured the disagreement directly:

```julia
# We do not export Mat_3from4. The point we need to make is empirical:
# at any state where the closure is non-linear (i.e. anywhere except
# the random-mixing IC), the per-shape Kirkwood RHS at order 4 differs
# from the uniform-anchor RHS, *even on the variables that order 3
# would track*. The package testset
# `@testset "Per-shape Kirkwood closure changes 4-vertex RHS"`
# pins this with a perturbed IC.
using NodeBasedModels: _build_sis_k3_m4_rhs, _build_sis_k3_m4_ic,
                       _build_variables, MotifClosure
cl_   = MotifClosure(3, 4)
shp_, vrs_, vidx_ = _build_variables(cl_, [:S, :I])
N_, ε_ = 500.0, 0.05
u0_   = _build_sis_k3_m4_ic(vidx_, N_, ε_,
                            2958.0/2, 2.0,
```

```
                                2958.0, 494.0, 6.0, 6.0, 0.0, 0.0)
rhs_kirk = _build_sis_k3_m4_rhs(vidx_; closure_kind = :kirkwood)
rhs_unif = _build_sis_k3_m4_rhs(vidx_; closure_kind = :uniform_anchor)
du_k = zeros(length(u0_)); du_u = zeros(length(u0_))
rng_p = MersenneTwister(424242)
u_pert = copy(u0_)
for v in vrs_
    iv = vidx_[(v.shape.name, v.state)]
    u_pert[iv] *= 0.4 + 1.2 * rand(rng_p)
end
rhs_kirk(du_k, u_pert, (β = 0.6, γ = 0.4), 0.0)
rhs_unif(du_u, u_pert, (β = 0.6, γ = 0.4), 0.0)
max_4v_diff = 0.0
for v in vrs_
    v.shape.n_nodes == 4 || continue
    iv = vidx_[(v.shape.name, v.state)]
    d  = abs(du_k[iv] - du_u[iv])
    d > max_4v_diff && (max_4v_diff = d)
end
println("Max |kirkwood - uniform-anchor| on 4-vertex RHS: ", max_4v_diff)
@assert max_4v_diff > 1e-3
```

```
Max |kirkwood - uniform-anchor| on 4-vertex RHS: 87.2932041618027
```

Two *different* equally-natural closure choices at order 4 (per-shape Kirkwood vs. a single-vertex anchor) produce *different* RHS values at a perturbed state. Whichever one we adopt, the rerouted order-3 RHS will disagree with the original. The fix attempt was abandoned not because of a coding bug but because no choice of order-4 closure can make the diagram commute simultaneously for all states — this is the content of the next two sections.

The Lean-certified obstruction

To be sure that the obstruction is structural rather than an artefact of the particular Kirkwood family we use, the package is paired with a formal verification in Lean 4 / Mathlib in `EdgeBasedModels.jl/proofs/EBCMCategory/`. There are three results:

T1 — Equivariance theorem (`MarginalisationFunctor.lean`)

For two closed systems $\dot{u}_4 = F_4(u_4)$ on V_4 and $\dot{u}_3 = F_3(u_3)$ on V_3, with continuous-linear $M : V_4 \to V_3$ and unique flows φ_4, φ_3, the trajectory identity

$$M\big(\varphi_4(u,t)\big) \;=\; \varphi_3\big(Mu,t\big) \quad \forall u,t$$

holds iff the infinitesimal identity

$$M \circ F_4 \;=\; F_3 \circ M$$

holds. The Lean theorem signature is

```
theorem dynamic_marginalisation_iff_equivariance
    (M : V₄ →L[ℝ] V₃) (F₄ : V₄ → V₄) (F₃ : V₃ → V₃)
    {φ₄ : V₄ → ℝ → V₄} {φ₃ : V₃ → ℝ → V₃}
    (h₄ : IsFlow F₄ φ₄) (h₃ : IsFlow F₃ φ₃)
    (uniq₃ : UniqueFlow F₃) :
    (∀ u, M (F₄ u) = F₃ (M u)) ↔ (∀ u t, M (φ₄ u t) = φ₃ (M u) t)
```

The contrapositive is the operational statement: failure of $M \circ F_4 = F_3 \circ M$ at a single point u is enough to *forbid* trajectory marginalisation. We do not need to integrate the ODE — a single snapshot of the RHS settles the question.

T2 — Concrete (2,1) -witness (`Obstructions.lean`)

The Lean witness is the smallest faithful Kirkwood mismatch:

- $V_4 = \mathbb{Q}^2$ with coordinates a, b.
- $V_3 = \mathbb{Q}^1$ with coordinate c.
- $M : V_4 \to V_3,\ M(u)(c) = u(a) + u(b)$.
- $F_4(u) = (u(a) \cdot u(b),\ u(b))$ — the bilinear $a \cdot b$ is the Kirkwood ratio in miniature.
- $F_3(v) = v(c)^2/4$ — the analogous Kirkwood-form closure on the coarse variable.

Theorem `kirkwood_obstruction_witness_value` proves over $\mathbb{Q}$:

$$M\big(F_4(1,3)\big)(c) \;-\; F_3\big(M(1,3)\big)(c) \;=\; 6 \;-\; 4 \;=\; 2.$$

Because the difference is an *exact rational* of magnitude 2, no floating-point noise can hide it.

T3a / b / c — Characterisation (`MarginalisationCharacterization.lean`)

T3a — `linear_closure_equivariant`: a *linear* closure family at order 4 yields an equivariant order-3 closure for free, whenever M intertwines the two linear pieces. Equivariance is therefore *easy* when the closure is linear.

T3b — `kirkwood_form_not_equivariant`: a closure family with a non-additive component (the Kirkwood form, abstracted as `IsKirkwoodForm`: $\exists u, v.\ C(u+v) \neq C(u) + C(v)$) cannot be equivariant for any choice of order-3 closure. The Lean proof is a one-line fibre-collapse argument:

- $u_1 = (1, 3)$ and $u_2 = (4, 0)$ both lie in the same M-fibre ($Mu_1 = Mu_2 = 4$).
- Equivariance would force $C_3(Mu_1) = C_3(Mu_2)$, hence $M(F_4 u_1) = M(F_4 u_2)$.
- But $M(F_4 u_1) = 1 \cdot 3 + 3 = 6$ while $M(F_4 u_2) = 4 \cdot 0 + 0 = 0$. Contradiction.

The reason no machinery is needed: the bilinear $a \cdot b$ is *killed* by the linear pushforward $a + b \mapsto c$. Any non-additive 4-vertex closure has the same fate — it is a feature of any Kirkwood form, not specific to a particular factorisation.

T3c — `kkr_necessary_not_sufficient`: even when the order-3 closure satisfies the Kiss–Kenah–Rempala pairwise-exactness criterion (e.g. $\kappa = 1$ for a Poisson degree distribution), the order-4 closure is *independent data*; if it is in Kirkwood form, T3b applies. KKR is necessary for order-3 to be self-exact, but says nothing about cross-order equivariance.

The Julia oracle: porting the Lean witness

The package testset `@testset "Lean-certified Kirkwood marginalisation obstruction"` ports the Lean (2,1) witness into Julia as a numerical regression. We reproduce it here:

```
M_witness = u → u[1] + u[2]            # (a, b) → c
F4_kirk   = u → (u[1] * u[2], u[2])    # (a, b) → (a · b, b)
F3_kirk   = v → v^2 / 4                # c → c² / 4

u_test = (1.0, 3.0)
Mu     = M_witness(u_test)
F4u    = F4_kirk(u_test)
lhs    = M_witness(F4u)                # M ∘ F₄
rhs_v  = F3_kirk(Mu)                   # F₃ ∘ M
diff_v = lhs - rhs_v

@printf("M(F₄ u)        = %.6f\n", lhs)
@printf("F₃(M u)        = %.6f\n", rhs_v)
@printf("difference     = %.6f  (Lean proves this is exactly 2)\n", diff_v)
@assert isapprox(diff_v, 2.0; atol = 1e-12)
```

```
M(F₄ u)        = 6.000000
F₃(M u)        = 4.000000
difference     = 2.000000  (Lean proves this is exactly 2)
```

```
# The fibre-collapse witness: u₁ = (1, 3), u₂ = (4, 0).
u1 = (1.0, 3.0)
u2 = (4.0, 0.0)
@printf("M u₁         = %.1f     M u₂         = %.1f\n",
        M_witness(u1), M_witness(u2))
@printf("M(F₄ u₁)     = %.1f     M(F₄ u₂)     = %.1f\n",
        M_witness(F4_kirk(u1)), M_witness(F4_kirk(u2)))
@assert M_witness(u1) == M_witness(u2)
@assert M_witness(F4_kirk(u1)) ≠ M_witness(F4_kirk(u2))
println("\nu₁ and u₂ live in the same M-fibre but are pushed to ",
        "different points by M ∘ F₄.\nNo F₃ can be a function of M u alone.")
```

```
M u₁         = 4.0     M u₂         = 4.0
M(F₄ u₁)     = 6.0     M(F₄ u₂)     = 0.0

u₁ and u₂ live in the same M-fibre but are pushed to different points by M ∘ F₄.
No F₃ can be a function of M u alone.
```

This snippet is mechanically equivalent to the Lean proof. A future edit that "fixes" Kirkwood marginalisation by tweaking constants will necessarily fail this test — and, by T1, will necessarily fail the trajectory marginalisation it sought to enforce.

Practical implications

The motif framework remains a powerful tool, but its use cases are narrower than one might initially hope.

Use motif models for:

- Inference when the network has structure that population models cannot represent — heterogeneous degree, clustering, specific motif abundances — and the goal is parameter estimation or trajectory fitting on a *fixed* graph. Closure error contributes a structural bias that is partly absorbed into the fit, and the explicit motif variables make it easy to introspect *which* correlations matter.
- Diagnostics. Even when $m = 4$ is no better than $m = 3$ for the marginal prevalence, the 4-vertex variables themselves carry interpretable information: the trajectory of $E_{C_4,\sigma}$ versus $E_{K_4,\sigma}$ tells you about local geometry that the Gillespie simulator does not summarise.

- Educational scaffolding. The hierarchy is the cleanest setting in which to *see* the closure error: each layer adds another set of variables, each closure formula is small and explicit, and the obstruction is a property of the formulas — not of opaque coefficients in a fitted model.

Do not use motif models for:

- Strict accuracy improvement by climbing m. The empirical separation on the random 3-regular host above (`err_m3 ≈ 0.026`, `err_m4 ≈ 0.348`) is a Lean-certified feature, not a tunable parameter. A larger m may be less accurate.
- Replacing the Gillespie ground truth in any setting where you have the budget to run it. The motif ODE is fast, but its bias is uncontrolled.

Triangle-rich hosts (where C_3, paw, $K_4 - e$, K_4 counts are non-trivial) are flagged in the testset comments as future work. T3b applies regardless of the host, but the *magnitude* of the non-monotonicity is a function of the host: a near-tree host like the random 3-regular will not provoke the worst behaviour, while a clustered host might either improve or worsen the gap. Empirical benchmarks on small-world / configuration-model graphs with tunable clustering remain to be done.

For monotone improvement one would need a closure family where T3a applies — i.e. a *linear* closure compatible with M. Possibilities include variational closures (e.g. maximum-entropy with linear moment constraints), Belief Propagation on the line graph, or the Dynamic Message Passing formalism, all of which sit outside the Kirkwood/Markov family used here. T3c warns that satisfying the KKR pairwise condition at order 3 is not enough.

Summary and connections

The motif framework completes the moment-closure ladder begun in vignette 03 and refined in vignette 09 (clustering and triangle-aware pair closures). Order m tracks every connected m-vertex induced subgraph by canonical state class; a Kirkwood-type ratio closes the $(m+1)$-vertex amplitudes; the singleton- and pair-marginal equations are exact summations of the higher-order transitions. The implementation is cross-validated by an independent Symbolics builder on every supported (k, m).

The capstone result is the Marginalisation Obstruction — Lean-certified theorems T1, T2, T3a/b/c — which prove that no Kirkwood-form closure can satisfy the equivariance required for $m+1$ to refine m as a strict moment hierarchy. The Julia oracle ports the Lean witness as a regression test that will fail loudly if anyone ever tries to "fix" Kirkwood marginalisation by parameter tweaking.

Vignette	Topic	Closure	Key result
03	Moment hierarchy	Independence; Kirkwood pair	Pair exact on trees

Vignette	Topic	Closure	Key result
07	Population pairwise	Bernoulli, Keeling	Mean-field epidemic threshold
09	Clustering	Keeling, Barnard, Eames	Triangles slow epidemics
11	Motif closures	Path-Kirkwood, per-shape	Marginalisation obstruction

Coming in Phase C: neighbourhood-based models that close at the *ego-network* level rather than at fixed-shape motifs, and cross-package comparisons against the EdgeBasedModels.jl message passing framework. Both will revisit the equivariance condition: the ego-network closure has additional symmetries that may circumvent T3b in restricted settings, and message passing is *exact* on locally tree-like hosts as $N \to \infty$, sidestepping the obstruction entirely by living in a different formal category.

Reading list

- `src/motif_based.jl` — the numeric motif framework (`MotifClosure`, `MotifShape`, `MotifVariable`, `MotifSystem`, `motif_based_sis`, `solve_motif`, `induced_subgraph_counts_4vertex`).
- `src/motif_symbolic.jl` — the Symbolics oracle (`build_motif_symbolic_rhs`).
- `test/runtests.jl` — testsets *"k=3 m=4 vs Gillespie on random 3-regular"*, *"Per-shape Kirkwood closure changes 4-vertex RHS"*, *"Lean-certified Kirkwood marginalisation obstruction"*, and *"Motif symbolic validator (B(c))"*.
- `EdgeBasedModels.jl/proofs/EBCMCategory/MarginalisationFunctor.lean` — Theorem T1, equivariance ? trajectory marginalisation.
- `EdgeBasedModels.jl/proofs/EBCMCategory/Obstructions.lean` — Theorem T2, concrete (2,1) $\mathbb{Q}$-witness.
- `EdgeBasedModels.jl/proofs/EBCMCategory/MarginalisationCharacterization.lean` — Theorems T3a/b/c, characterisation of equivariant closures.
- `EdgeBasedModels.jl/proofs/EBCMCategory/MARGINALISATION_SPEC.md` — the up-front specification document, useful for orientation before reading the proofs.

NetworkOutbreaks SSA ribbon

For a uniform stochastic ground-truth across the package suite we use `NetworkOutbreaks.jl`'s Gillespie SSA. Where the deterministic prediction in this vignette already sits inside the SSA mean $\pm$ 1σ ribbon — see vignette `01_sir_on_graphs` for the canonical overlay pattern — we omit the redundant ribbon here for clarity.

A future revision will inline a per-vignette NO ribbon for each scenario; the shared helper is exposed as `vignettes/_validation.jl#gillespie_ribbon` and applied in vignette 01.